

Recursividad y Manejo de Memoria Dinámica

Semana 9 — Definir lo grande en términos de lo pequeño: pila de ejecución, garbage collector y la naturaleza recursiva del lenguaje

1. ¿Por qué estudiar la recursividad?

Hasta esta semana, todos nuestros algoritmos han sido *iterativos*: una secuencia de instrucciones, eventualmente repetida con un `while` o un `for`, que avanza paso a paso hacia una solución. La iteración es una forma natural de pensar para la máquina —refleja directamente el ciclo *fetch-decode-execute* que estudiamos en la Semana 1— pero no siempre es la forma más natural de pensar para nosotros, ni la más clara para el lector del programa.

Hay problemas cuya estructura es **intrínsecamente recursiva**: la respuesta para una entrada de tamaño n se define en términos de la respuesta para una entrada de tamaño menor. Tres ejemplos cotidianos lo ilustran:

- **El factorial.** La definición matemática es explícitamente recursiva: $n! = n \cdot (n-1)!$, con caso base $0! = 1$. Forzarla a una forma iterativa es posible, pero requiere introducir un acumulador y un índice que la definición original no necesita.
- **La búsqueda binaria.** En la Semana 5 la implementamos iterativamente. Pero su descripción natural es recursiva: *buscar en una lista ordenada equivale a comparar con el medio y, si no es igual, buscar en una de las dos mitades*. Mismo problema, mitad del tamaño.
- **Recorrer un sistema de archivos.** Un directorio contiene archivos y otros directorios; cada uno de esos otros directorios es a su vez un directorio que contiene archivos y otros directorios. La estructura es recursiva, y el algoritmo que la recorre lo es también con naturalidad.

La **recursividad** es una técnica de diseño algorítmico en la que una función se define *en términos de sí misma*, aplicada a una entrada estrictamente más pequeña. No es solo un truco sintáctico de Python: es una forma de pensar que se vuelve indispensable cuando los datos tienen estructura jerárquica (árboles,

expresiones aritméticas, sistemas de archivos), cuando el problema admite la estrategia *divide y vencerás* (Semana 6), o cuando la definición matemática del problema es ella misma recursiva.

Conexión con semanas anteriores

En la **Semana 4** presentamos las funciones y la idea de que cada llamada crea su propio *stack frame*; en la **Semana 6** aplicamos «divide y vencerás» al merge sort y al quicksort, donde cada subproblema era una sublista a procesar; en la **Semana 7** estudiamos en detalle la estructura de la pila de ejecución —los *stack frames*, la dirección de retorno, las variables locales— y los cuatro segmentos del modelo de memoria (text, data, heap, stack); en la **Semana 8** mencionamos que un archivo binario puede contener índices que apuntan a otros bloques del mismo archivo. Todos esos hilos se anudan esta semana: la recursividad es la idea de funciones de la Semana 4 llevada al límite, ejecutada sobre la pila de la Semana 7, aplicada a problemas con estructura jerárquica.

Pero la recursividad no viene gratis. Cada llamada recursiva consume un *stack frame*; la pila tiene un tamaño limitado; los lenguajes que no la optimizan —Python entre ellos— pagan un costo en cada llamada que la versión iterativa no paga. Esta semana, entonces, no solo aprenderemos *cómo* escribir algoritmos recursivos, sino también *cuándo* elegirlos —y cuándo preferir la iteración.

El segundo gran tema de la semana es la **memoria dinámica**. En la Semana 7 vimos que los objetos de Python viven en el *heap*; lo que no discutimos allí fue quién los libera cuando ya no se usan. En lenguajes como C o C++ esa responsabilidad recae sobre el programador: cada `malloc` debe tener su `free` correspondiente, y olvidarse de liberar produce *memory leaks* que terminan agotando la RAM. Python —como Java, C# o Go— ofrece un **garbage collector**: un mecanismo automático que detecta los objetos sin referencias activas y los elimina sin intervención del programador. Comprender cómo funciona el GC de Python es importante por dos razones: explica por qué ciertas prácticas (como las referencias circulares) pueden tener consecuencias sutiles, y nos prepara para apreciar el costo y el diseño de los lenguajes que *sí* nos hacen liberar la memoria a mano.

Cerraremos con una conexión holística que retoma directamente la naturaleza recursiva: las **gramáticas formales**. Los lenguajes de programación se describen

mediante reglas recursivas (BNF, EBNF), y los compiladores e intérpretes que estudiamos en la Semana 6 construyen *árboles de derivación* aplicando esas reglas. Veremos que el código Python que escribimos es procesado por un programa —el parser de CPython— cuya tarea es esencialmente recursiva.

2. Anatomía de un algoritmo recursivo

Todo algoritmo recursivo bien diseñado tiene **dos componentes obligatorios**:

Caso base y caso recursivo

Las dos piezas indispensables de toda recursión correcta

- **Caso base.** Una o más entradas tan pequeñas que la respuesta se conoce sin necesidad de hacer más llamadas. Es el *punto de detención* de la recursión. Sin caso base, la función se llamaría a sí misma indefinidamente: la pila crecería hasta agotarse y Python lanzaría un `RecursionError`.
- **Caso recursivo.** Para entradas mayores que el caso base, la función delega el trabajo en sí misma sobre una entrada estrictamente más pequeña y combina el resultado con un trabajo local trivial. Es el *paso inductivo* que reduce gradualmente el problema hacia el caso base.

A esto se añade una condición más sutil pero igual de importante: cada llamada recursiva debe acercarse *de manera demostrable* al caso base. Si la entrada se reduce en cada paso ($n \rightarrow n-1$, o lista $\rightarrow$ resto, o intervalo $\rightarrow$ mitad del intervalo), la convergencia está garantizada. Si no, tenemos una recursión infinita disfrazada.

2.1. El factorial: la recursión más pura

El factorial es el ejemplo canónico para introducir la recursividad porque su definición matemática ya es recursiva. Recordemos:

$$n! = 1 \quad \text{si } n = 0$$

$$n! = n \cdot (n - 1)! \quad \text{si } n > 0$$

La línea de arriba es el caso base. La línea de abajo es el caso recursivo. Traducir esta definición a un programa Python es casi una transcripción.

Análisis (Pólya)

Antes de programar, sigamos los cuatro pasos de Pólya. Comprender el problema: dado un entero no negativo n , calcular el producto $1 \cdot 2 \cdot 3 \cdots n$. Por convención, $0! = 1$ (producto vacío). Casos de análisis:

Entrada	Salida esperada	Tipo de caso
$n = 0$	1	Límite (caso base)
$n = 1$	1	Límite (primer caso recursivo trivial)
$n = 5$	120	Normal
$n = 10$	3628800	Normal
$n = -3$	Indefinido	Fuera de dominio $\rightarrow$ precondition

Diseño (pseudocódigo)

```
Algoritmo factorial(n)
    // Precondición:  $n \geq 0$  y  $n$  entero
    si  $n = 0$ :
        retornar 1    // caso base
    sino:
        retornar  $n \cdot \text{factorial}(n - 1)$     // caso recursivo
```

Codificación (Python)

```
def factorial(n):
    ...
```

SECCIÓN DECLARATIVA

Descripción: Calcula el factorial de n de forma recursiva.

$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$, con la convención $0! = 1$

Precondición: n es un entero no negativo ($n \geq 0$).

Postcondición: retorna el valor de $n!$ como un entero.

'''

--- SECCIÓN ALGORÍTMICA ---

Caso base: $0! = 1$ por convención (producto vacío)

if $n == 0$:

return 1

Caso recursivo: $n! = n \cdot (n-1)!$

return $n * \text{factorial}(n - 1)$

La función no tiene prólogo ni epílogo claros: cada invocación hace exactamente *una* cosa, y la decisión entre «devolver 1» y «multiplicar por la subllamada» agota toda la lógica. Esta es una característica típica de las funciones recursivas puras: son tan compactas que la división en prólogo / resolución / epílogo se difumina, porque la *resolución misma* es delegar al subproblema.

Lectura literaria del código

Una función recursiva bien escrita se lee como su definición matemática: «*el factorial de cero es uno; el factorial de n , para n positivo, es n por el factorial de n menos uno*». Si la definición del problema es naturalmente recursiva, el código recursivo es más próximo a la *especificación* y, por tanto, más fácil de auditar y de demostrar correcto. Esta proximidad entre especificación y código es una de las grandes virtudes pedagógicas de la recursividad.

2.2. La omisión del caso base: el peligro silencioso

¿Qué ocurre si olvidamos el caso base?

```
def factorial_roto(n):  
    # Faltó el caso base. Este programa NO termina por sí solo.  
    return n * factorial_roto(n - 1)
```

```
factorial_roto(5)  
# RecursionError: maximum recursion depth exceeded
```

La función se llama a sí misma con argumento 4, que se llama con 3, que se llama con 2, que se llama con 1, que se llama con 0, que se llama con -1 , que se llama con -2 ... Sin caso base, el argumento decrece eternamente y nunca se devuelve un valor. Cada llamada apila un nuevo *stack frame*; la pila crece hasta que Python detecta que se superó el límite (por defecto, 1000 frames anidados) y aborta con `RecursionError`. Volveremos sobre este límite y cómo se relaciona con la pila del sistema operativo en la Sección 7.

También es importante que el caso base *esté correctamente alineado con la precondition*. Si la precondition es $n \geq 0$, el caso base $n = 0$ es alcanzable desde cualquier entrada válida. Pero si por error escribiéramos:

```
def factorial_inalcanzable(n):  
    if n == 1:                # caso base equivocado: no cubre n = 0  
        return 1  
    return n * factorial_inalcanzable(n - 1)
```

```
factorial_inalcanzable(0)  
# 0 → -1 → -2 → ... → RecursionError
```

Para $n = 0$, el caso base $n == 1$ nunca se alcanza: la recursión entra en valores negativos y diverge. Esta clase de error es traicionera porque la función *parece* recursiva bien escrita y funciona perfecto para $n \geq 1$.

Tres preguntas que toda recursión debe responder

1. ¿**Hay caso base?** — sin él, la recursión no termina nunca.
2. ¿**Es alcanzable** el caso base desde cualquier entrada que cumpla la precondition? — un caso base inalcanzable equivale a no tenerlo.
3. ¿**Cada llamada recursiva** usa una entrada estrictamente más próxima al caso base? — si la entrada no decrece, la recursión diverge aunque el caso base exista y sea alcanzable.

3. La pila de ejecución y el árbol de recursión

Para entender de verdad qué pasa cuando llamamos a `factorial(5)`, hay que volver al modelo de memoria de la Semana 7. Cada invocación de función — recursiva o no— **apila un nuevo *stack frame*** sobre la pila de ejecución del proceso. Ese frame contiene los parámetros de la llamada, las variables locales, y la **dirección de retorno** a la que se debe saltar cuando la función termine.

En una función recursiva, las llamadas se anidan: la función A llama a A', que llama a A'', y así sucesivamente. Cada una está *en pausa*, esperando que su subllamada devuelva un valor para poder continuar. La pila guarda toda esa secuencia de invocaciones pendientes.

3.1. Traza con `factorial(5)`

Sigamos la ejecución paso a paso. Cuando se invoca `factorial(5)`, ocurren las siguientes apilaciones:

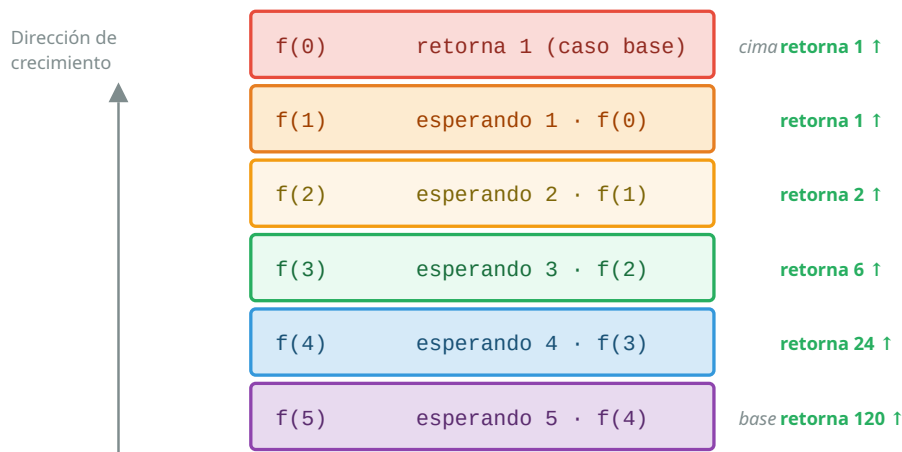
Paso	Acción	Pila (de la base hacia arriba)	Valor pendiente
1	Llamada <code>factorial(5)</code>	[<code>f(5)</code>]	$5 \cdot f(4)$
2	<code>f(5)</code> llama a <code>f(4)</code>	[<code>f(5)</code> , <code>f(4)</code>]	$4 \cdot f(3)$
3	<code>f(4)</code> llama a <code>f(3)</code>	[<code>f(5)</code> , <code>f(4)</code> , <code>f(3)</code>]	$3 \cdot f(2)$
4	<code>f(3)</code> llama a <code>f(2)</code>	[<code>f(5)</code> , <code>f(4)</code> , <code>f(3)</code> , <code>f(2)</code>]	$2 \cdot f(1)$
5	<code>f(2)</code> llama a <code>f(1)</code>	[<code>f(5)</code> , <code>f(4)</code> , <code>f(3)</code> , <code>f(2)</code> , <code>f(1)</code>]	$1 \cdot f(0)$

Paso	Acción	Pila (de la base hacia arriba)	Valor pendiente
6	f(1) llama a f(0); caso base	[f(5), f(4), f(3), f(2), f(1), f(0)]	retorna 1
7	f(1) recibe 1, calcula 1·1, retorna 1	[f(5), f(4), f(3), f(2), f(1)]	retorna 1
8	f(2) recibe 1, calcula 2·1, retorna 2	[f(5), f(4), f(3), f(2)]	retorna 2
9	f(3) recibe 2, calcula 3·2, retorna 6	[f(5), f(4), f(3)]	retorna 6
10	f(4) recibe 6, calcula 4·6, retorna 24	[f(5), f(4)]	retorna 24
11	f(5) recibe 24, calcula 5·24, retorna 120	[f(5)]	retorna 120
12	fin	[]	resultado: 120

Observen las dos fases marcadas: una de **apilamiento** en la que cada llamada delega en la siguiente sin haber terminado su propio cálculo, y una de **desapilamiento** en la que las llamadas se completan en orden inverso, cada una usando el valor que devolvió la inmediatamente anterior. La pila alcanza una profundidad máxima de 6 frames (uno por cada valor de n entre 5 y 0), y todo el cómputo «real» ocurre en el camino de vuelta.

3.2. Visualización de la pila durante la fase de apilamiento

Pila de ejecución en el momento del caso base de factorial(5)



Las llamadas inferiores están en pausa, esperando el valor de la inmediatamente superior

🔗 Conexión con Semana 7: stack frames y dirección de retorno

Cada caja en la figura es un *stack frame*, esa estructura que estudiamos en detalle en la Semana 7. Cada frame contiene: los parámetros (en este caso, el valor de n de esa invocación), las variables locales (en factorial no hay), y la **dirección de retorno** —la instrucción a la que la CPU debe saltar cuando esa invocación termine. La dirección de retorno de $f(0)$ apunta a la línea `return n * factorial(n - 1)` dentro de $f(1)$, justo después del `factorial(n - 1)`; la de $f(1)$ apunta al mismo lugar, pero dentro del frame de $f(2)$; y así sucesivamente. La *misma línea de código* aparece referenciada desde múltiples frames; lo que cambia es el contexto (qué valor de n y qué multiplicación está pendiente).

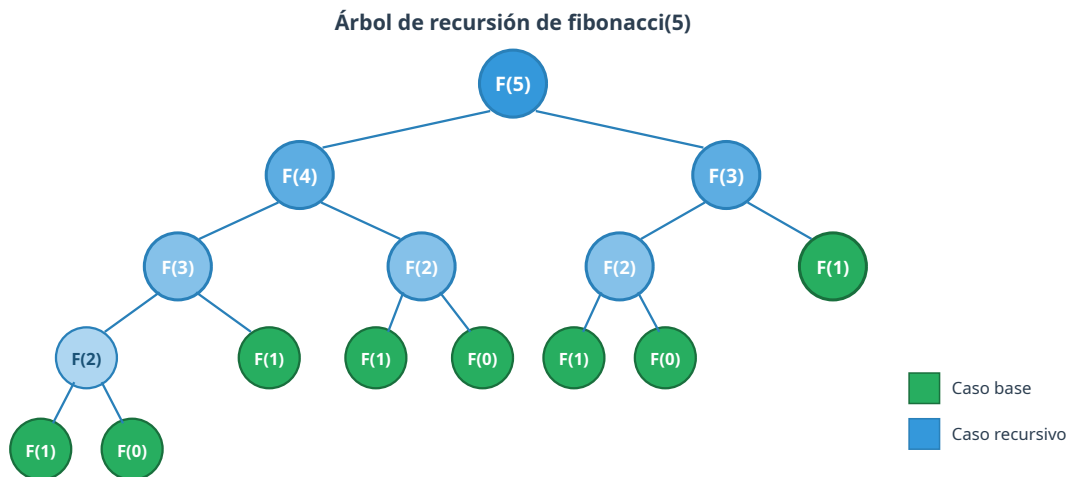
3.3. El árbol de recursión: una mirada complementaria

El factorial es una recursión *lineal*: cada llamada hace una sola subllamada. Su «árbol de recursión» es por tanto una cadena: cada nodo tiene un solo hijo, y la altura del árbol coincide con la profundidad máxima de la pila. Pero hay recursiones que ramifican: cada llamada hace dos o más subllamadas. El ejemplo clásico es **Fibonacci**:

$$F(0) = 0 \quad F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2), \quad n \geq 2$$

Para calcular $F(5)$, la función se ramifica en $F(4)$ y $F(3)$; cada uno se ramifica en otros dos; y así sucesivamente hasta los casos base. El árbol de recursión se ve así:



F(2) se calcula 3 veces, F(1) 5 veces, F(0) 3 veces — trabajo redundante creciente

El árbol expone un problema serio de la implementación recursiva ingenua de Fibonacci: hay **recomputación masiva**. $F(2)$ se calcula tres veces, $F(1)$ cinco veces. La cantidad de llamadas crece exponencialmente con n : para calcular $F(40)$ la función se invoca más de 200 millones de veces. Es un caso de libro donde la elegancia matemática de la definición recursiva esconde un costo computacional inaceptable. Las soluciones a este problema (la *memoización*, la programación dinámica, la recursión con acumulador) son tema de la Semana 12. Por ahora basta con saber: *tener un árbol de recursión con muchos subproblemas repetidos es una señal de alarma*.

Pila vs. árbol: dos perspectivas, dos costos

El **árbol de recursión** mide el *número total de llamadas* —es decir, el costo en **tiempo**. La **pila**, en cambio, mide la *cantidad máxima de frames simultáneamente activos* —el costo en **memoria**. Para Fibonacci, el árbol tiene un número exponencial de nodos, pero la pila nunca supera n niveles, porque los hijos se procesan uno después del otro y solo el camino actual desde la

raíz hasta el nodo en proceso está en la pila. Confundir las dos métricas es un error frecuente; mantenerlas distintas es esencial para analizar bien el costo de un algoritmo recursivo.

4. Iteración vs. recursión: criterios de decisión

Toda función recursiva puede escribirse iterativamente, y viceversa —es un resultado clásico de teoría de la computación: ambos formalismos son equivalentes en poder expresivo. La pregunta interesante no es *¿se puede?* sino *¿cuál conviene en cada caso?*

Comparemos las dos versiones del factorial:

```
# Versión recursiva: 5 líneas, traduce literalmente la definición
def factorial_rec(n):
    if n == 0:
        return 1
    return n * factorial_rec(n - 1)
```

```
# Versión iterativa: 5 líneas, requiere acumulador y bucle explícito
def factorial_iter(n):
    resultado = 1
    for i in range(2, n + 1): # produce 2, 3, ..., n
        resultado *= i
    return resultado
```

Las dos producen el mismo resultado, pero pagan costos diferentes y exigen al lector estilos de comprensión distintos. La versión recursiva pide al lector razonar por *inducción* («si funciona para $n-1$, funciona para n »); la iterativa pide razonar por *invariantes* («al término de la iteración i , resultado contiene $i!$ »).

4.1. Tabla comparativa

Dimensión	Recursión	Iteración
Legibilidad	Alta cuando el problema es naturalmente recursivo (estructuras jerárquicas, divide y vencerás, definiciones inductivas).	Alta cuando el problema es naturalmente iterativo (recorrer una secuencia, acumular, contar).
Memoria (pila)	Consume $O(\text{profundidad})$ <i>stack frames</i> . Para $n = 10\,000$ puede agotar la pila.	Consume $O(1)$ en variables locales del único frame activo. Independiente de n .
Tiempo	Costo extra por cada llamada: crear frame, copiar parámetros, saltar y volver. Típicamente 1.5-3× más lento que la versión iterativa equivalente en Python.	Sin sobrecarga de invocación; el bucle se traduce a saltos eficientes.
Riesgo de divergencia	Caso base ausente o inalcanzable $\Rightarrow$ <code>RecursionError</code> .	Condición de salida ausente o errónea $\Rightarrow$ bucle infinito; el programa cuelga.
Demostración de correctitud	Por inducción estructural. Suele ser corta y elegante.	Por invariante de bucle. Requiere identificar la propiedad y demostrarla en tres pasos (inicialización, conservación, terminación).
Optimización del compilador	Lenguajes funcionales (Scheme, Haskell) optimizan tail recursion a iteración. Python no .	Es la forma natural; los compiladores la optimizan agresivamente.

4.2. Cuándo elegir cada una

La elección entre recursión e iteración no debe basarse en gusto personal sino en la **estructura del problema**. Tres criterios prácticos:

Criterio 1: ¿Los datos son jerárquicos?

Si la entrada es un árbol, un sistema de archivos, una expresión con paréntesis anidados, un JSON con objetos dentro de objetos, o una página HTML con etiquetas dentro de etiquetas, la recursión es casi siempre la elección correcta. La estructura recursiva del dato hace que la versión iterativa requiera mantener manualmente una pila auxiliar —que es exactamente lo que la versión recursiva obtiene gratis del lenguaje.

Criterio 2: ¿El problema se resuelve por «divide y vencerás»?

Si dividimos la entrada en partes, resolvemos cada parte por separado, y combinamos los resultados —como hicimos en merge sort y quick sort en la Semana 6— la formulación recursiva es la natural. La versión iterativa de merge sort existe (bottom-up), pero es bastante menos legible.

Criterio 3: ¿La profundidad puede ser grande?

Si la profundidad de la recursión puede crecer mucho con el tamaño de la entrada (por ejemplo, recorrer una lista de 100 000 elementos uno por uno), la recursión es *peligrosa* en Python: tarde o temprano agotará la pila. En este caso la iteración es preferible aunque la formulación parezca más torpe.

El criterio 3 es importante: el factorial recursivo, por elegante que sea, falla para $n > 1000$ en Python con la configuración por defecto. El factorial es un mal candidato para la recursión *en producción*; es un excelente candidato para la recursión *en la enseñanza*, porque su simplicidad lo hace ideal para entender el mecanismo. Esa distinción entre «recursión didáctica» y «recursión productiva» conviene tenerla siempre presente.

El límite de recursión de Python

Por defecto, Python aborta con `RecursionError` al alcanzar 1000 frames anidados. El valor exacto puede consultarse y modificarse mediante `sys.getrecursionlimit()` y `sys.setrecursionlimit(N)`. Subirlo es *posible* pero *peligroso*: el límite está calibrado para no agotar la pila **real** del sistema operativo (típicamente 8 MiB en Linux). Subirlo demasiado puede causar un *stack overflow* que aborta el intérprete por completo, sin posibilidad de recuperación. La estrategia recomendada para problemas que requieren mucha recursión no es subir el límite, sino reescribir el algoritmo iterativamente o convertir la recursión a una pila explícita en el heap.

5. Patrones recursivos canónicos

Hay un pequeño conjunto de patrones recursivos que aparecen una y otra vez. Reconocerlos y dominarlos es el camino para luego diseñar recursiones propias con confianza. Veremos tres: *recorrer* y *acumular* (suma de lista), *dividir el problema en mitad* (potencia rápida y búsqueda binaria recursiva), y *generalizar el segundo a otros divide y vencerás*.

5.1. Suma de los elementos de una lista

Calcular la suma de una lista parece tarea de iteración. Y lo es. Pero conviene escribirla recursivamente al menos una vez, para fijar el patrón «*el primero, más la suma del resto*».

```
Algoritmo suma(lista, i)
    // Precondición: lista es una secuencia indexable de
    números;  $0 \leq i \leq \text{len}(\text{lista})$ 
    si i = longitud(lista):
        retornar 0    // suma de la lista vacía
    sino:
        retornar lista[i] + suma(lista, i + 1)
```

```
def suma(lista, i=0):
    '''
    Descripción: Suma recursivamente los elementos de lista desde
        hasta el final.
    Precondición: lista es una secuencia indexable de números; 0 <
    Postcondición: retorna lista[i] + lista[i+1] + ... + lista[ler
        retorna 0 si i == len(lista) (suma vacía).
    '''

    # Caso base: posición pasada el final → suma vacía
    if i == len(lista):
        return 0

    # Caso recursivo: el elemento actual más la suma del resto
    return lista[i] + suma(lista, i + 1)
```

Notas pedagógicas sobre esta solución:

- El **parámetro i con valor por defecto** permite invocar `suma(datos)` sin tener que pasar el cero inicial. La función acepta una llamada externa simple y se llama internamente con el índice avanzado.
- El **caso base** es `i == len(lista)`, no `i > len(lista)`: con el operador de igualdad la equivalencia es exacta y elimina cualquier riesgo de pasarse de largo.
- La profundidad de la recursión es n : este algoritmo no escala a listas grandes en Python. Es didáctico, no productivo. Para sumar listas reales usamos `sum(lista)`, que internamente itera.

5.2. Potencia rápida: la recursión logarítmica

Calcular x^n tiene una versión recursiva ingenua de profundidad $O(n)$:

```
Algoritmo potencia_lenta(x, n)
    si n = 0:
        retornar 1
    sino:
```

```
retornar x · potencia_lenta(x, n - 1)
```

Pero hay una observación inteligente: si n es par, $x^n = (x^{n/2})^2$; si n es impar, $x^n = x \cdot x^{n-1}$. Esto reduce el problema a la mitad en cada paso par, dando una profundidad $O(\log n)$ en lugar de $O(n)$. Es la **exponenciación rápida**:

Algoritmo potencia(x, n)

```
// Precondición: n es un entero no negativo
```

```
si n = 0:
```

```
    retornar 1
```

```
si n es par:
```

```
    mitad ← potencia(x, n >> 1)
```

```
    retornar mitad · mitad
```

```
sino:
```

```
    retornar x · potencia(x, n - 1)
```

```
def potencia(x, n):
    '''
    Descripción: Calcula x elevado a n por exponenciación rápida
                  (recursión binaria con división por 2).
    Precondición: n es un entero no negativo (n >= 0); x es número
    Postcondición: retorna x ** n. Profundidad de recursión: O(log
    '''

    # Caso base:  $x^0 = 1$  por definición
    if n == 0:
        return 1

    # Caso recursivo par:  $x^n = (x^{(n/2)})^2$ ; con  $n \gg 1$  conectamos
    if n % 2 == 0:
        mitad = potencia(x, n >> 1)
        return mitad * mitad

    # Caso recursivo impar:  $x^n = x \cdot x^{(n-1)}$ 
    return x * potencia(x, n - 1)
```

¿Por qué $n \gg 1$ y no $n // 2$?

Para enteros no negativos, las dos expresiones producen el mismo resultado. Pero $n \gg 1$ es un *desplazamiento de bits a la derecha*: una operación elemental de la ALU de la CPU, traducida a una sola instrucción de máquina. La división entera $n // 2$ es una operación más costosa en hardware. Esta preferencia — que mantenemos desde la Semana 1 — tiene más valor pedagógico que práctico en Python (donde la diferencia se diluye en el costo del intérprete), pero en C, en sistemas embebidos o en procesamiento de señales sí importa. Documentar la operación con un comentario, como hicimos arriba, es lo que distingue al uso intencional del uso oscuro.

Para entender el ahorro, comparemos el *número de multiplicaciones* requerido para calcular x^{20} :

Versión	Llamadas recursivas	Multiplicaciones	Profundidad de pila
Lenta — $O(n)$	20	20	20
Rápida — $O(\log n)$	6 (20→10→5→4→2→1→0)	6	6

Para $n = 1\,000\,000$ la diferencia es drástica: 1 millón vs. 20. La exponenciación rápida es la base de muchos algoritmos importantes en criptografía moderna (RSA, Diffie-Hellman) donde se calculan potencias modulares de enteros gigantes. Es uno de los lugares donde elegir bien el algoritmo es la diferencia entre «tarda un microsegundo» y «tarda más que la vida del universo».

5.3. Búsqueda binaria recursiva: revisita de la Semana 5

En la Semana 5 implementamos la búsqueda binaria iterativamente, con un bucle que movía los extremos *izq* y *der* hacia el medio. La formulación recursiva es más próxima a la idea original: *buscar en una lista ordenada equivale a comparar con el medio y, si no es igual, buscar en una de las dos mitades*.

```

Algoritmo busqueda_binaria_rec(datos, clave, izq, der)
    // Precondición: datos está ordenado en forma no
    decreciente
    si izq > der:
        retornar -1    // caso base 1: intervalo vacío — no
    encontrado
    medio ← (izq + der) >> 1
    si datos[medio] = clave:
        retornar medio    // caso base 2: encontrado
    si datos[medio] < clave:
        retornar busqueda_binaria_rec(datos, clave, medio + 1,
    der)
    sino:
        retornar busqueda_binaria_rec(datos, clave, izq, medio

```

```
def busqueda_binaria_rec(datos, clave, izq=0, der=None):  
    '''  
    Descripción: Busca clave en datos[izq..der] mediante recursión  
        Cada llamada descarta la mitad del intervalo, dando comple  
    Precondición: datos está ordenada en forma no decreciente; cla  
        con los elementos de datos mediante < y ==.  
    Postcondición: retorna el índice i tal que datos[i] == clave,  
        o -1 si clave no pertenece a datos.  
    '''  
  
    # Inicialización del intervalo en la primera invocación exterr  
    if der is None:  
        der = len(datos) - 1  
  
    # Caso base 1: intervalo vacío → no está  
    if izq > der:  
        return -1  
  
    # Cálculo del punto medio con desplazamiento de bits (cf. Sema  
    medio = (izq + der) >> 1  
  
    # Caso base 2: encontrado en el medio  
    if datos[medio] == clave:  
        return medio  
  
    # Caso recursivo: descender en la mitad correspondiente  
    if datos[medio] < clave:  
        return busqueda_binaria_rec(datos, clave, medio + 1, der)  
    return busqueda_binaria_rec(datos, clave, izq, medio - 1)
```

Esta versión tiene **dos casos base**: el intervalo vacío (no encontrado) y el acierto en el medio (encontrado). El caso recursivo se bifurca: ir a la mitad derecha si la clave es mayor, o a la izquierda si es menor. La profundidad de recursión es $O(\log n)$: para una lista de un millón de elementos, no excede los 20 frames. Aquí la

recursión sí es productivamente segura.

5.4. Comparación: recursividad lineal vs. logarítmica

Algoritmo	Profundidad de pila	Para $n = 10^6$	¿Recursión productiva en Python?
Factorial recursivo	$O(n)$	10^6 frames	No — agota la pila
Suma recursiva de lista	$O(n)$	10^6 frames	No — agota la pila
Potencia rápida	$O(\log n)$	~20 frames	Sí
Búsqueda binaria recursiva	$O(\log n)$	~20 frames	Sí
Merge sort recursivo (Sem. 6)	$O(\log n)$	~20 frames	Sí

El patrón se hace claro: las recursiones *logarítmicas*, donde cada paso reduce el problema a una fracción constante (mitad, tercio, etc.), son seguras y eficientes en Python. Las recursiones *lineales*, donde cada paso reduce el problema en una unidad, son didácticas pero peligrosas en producción.

6. Recursividad de cola y la política de Python

Una recursión se llama **de cola** (*tail recursion*) cuando la **llamada recursiva es la última operación** que la función realiza antes de retornar. Es decir, la función no tiene nada más que hacer después de recibir el valor de la subllamada: simplemente lo devuelve.

Comparemos las dos versiones del factorial:

```
# NO es de cola: después de la llamada recursiva todavía hay que r
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)    # ← multiplicación pendiente: NC

# Sí es de cola: la llamada recursiva es lo último que se hace
def factorialCola(n, acumulador=1):
    if n == 0:
        return acumulador
    return factorialCola(n - 1, n * acumulador)    # ← nada pendiente
```

La versión de cola usa un **parámetro acumulador** que carga el resultado parcial. Cuando llega el caso base, el acumulador ya contiene el factorial completo y se devuelve directamente. No hay nada por hacer en el camino de vuelta: cada llamada toma el resultado de la siguiente y lo retorna sin tocarlo.

6.1. Por qué la recursión de cola es teóricamente eficiente

En una llamada de cola, el frame de la función actual es *inútil*: no tiene más cálculos que hacer después del retorno de la subllamada. Un compilador inteligente puede observar esto y, en lugar de apilar un nuevo frame, **reutilizar el frame actual**: descartar las variables locales viejas, escribir las nuevas en el mismo lugar, y saltar al inicio de la función. Esto se llama *tail call optimization* (TCO) o *tail call elimination*, y convierte la recursión de cola en un bucle disfrazado, sin crecimiento de pila.

Los lenguajes de la familia funcional —Scheme, Standard ML, OCaml, Haskell— *garantizan* esta optimización: en ellos, escribir un bucle es escribir una recursión de cola, y no hay penalización. Los compiladores modernos de C, C++ y Rust también la aplican como optimización agresiva cuando se invocan con un nivel de optimización alto (por ejemplo, la opción `-O2` de `gcc` o `clang`, que activa optimizaciones estándar de producción entre las que se incluye la eliminación de llamadas de cola). Java la aplica parcialmente. Python **no** la aplica, ni la aplicará.

6.2. Por qué Python rechaza la TCO

La decisión de no implementar TCO en Python es un caso interesante de *filosofía de diseño de lenguaje*. Guido van Rossum, creador del lenguaje, publicó en 2009 una entrada de blog donde lista varias razones que siguen siendo válidas hoy:

- **Trazabilidad de errores.** Cuando una excepción explota, el *traceback* de Python lista los frames de todas las llamadas anidadas. La TCO destruiría esa información: solo se vería el frame final, sin el camino de invocaciones que llevó al error. Para un lenguaje que privilegia la legibilidad y la depurabilidad, este costo es inaceptable.
- **Filosofía no-funcional.** Python es un lenguaje predominantemente imperativo. La iteración con `for` y `while` es idiomática y eficiente; usar la recursión de cola para simular un bucle sería una práctica forzada y poco pythonista.
- **Sutilezas semánticas.** Detectar automáticamente cuándo una llamada es realmente «de cola» en Python es complicado por la presencia de decoradores, contextos `with`, manejadores de excepciones, y otras estructuras que pueden requerir trabajo después del `return`.

El consenso de la comunidad Python es claro: *si una recursión es lineal y su versión iterativa es viable, se escribe iterativa*. La recursión de cola en Python existe como concepto teórico y como ejercicio mental, pero no como herramienta productiva para evitar el crecimiento de pila.

Lo que la recursión de cola sí nos enseña

Aunque Python no aproveche la optimización, escribir una versión de cola de un algoritmo recursivo es un ejercicio formativo. Obliga a identificar el *estado parcial* del cómputo y a transportarlo explícitamente como parámetro. Esa misma transformación —de «recursión con valor pendiente» a «recursión con acumulador»— es exactamente la traducción a iteración: el acumulador se vuelve la variable del bucle, y la recursión se vuelve un `while`. Por eso decimos que pensar en términos de recursividad de cola es un puente conceptual hacia la versión iterativa correcta.

Comparemos la versión de cola con la versión iterativa, lado a lado:

```
# Recursión de cola
def factorialCola(n, acum=1):
    if n == 0:
        return acum
    return factorialCola(n - 1, n * acum)

# Versión iterativa equivalente: misma estructura, sin recursión
def factorial_iter(n):
    acum = 1
    while n != 0:
        acum = n * acum      # lo que era el segundo argumento de l
        n = n - 1           # lo que era el primer argumento de la
    return acum
```

Las dos funciones son *matemáticamente equivalentes*. La iterativa es lo que la recursiva de cola se convertiría tras aplicarle TCO. En Python, la versión iterativa es la que efectivamente debemos escribir cuando queremos eficiencia y profundidad ilimitada.

6.3. Una nota sobre el uso de break

En el código iterativo de arriba se podría haber escrito el bucle con `while True` y un `break` dentro del `if n == 0`. En esta asignatura preferimos la condición directa en el encabezado del `while` —`while n != 0:`— porque expresa de manera más clara *cuándo* termina la iteración: la condición de continuación está donde el lector la espera. La estructura sin `break` hace explícita la **postcondición del bucle**: al salir, sabemos que `n == 0`, y por tanto `acum` es el factorial buscado. Cuando hace falta combinar varias condiciones de salida, el operador `and` con cortocircuito y la inspección post-bucle resuelven el caso, como vimos en la Semana 3 con la búsqueda lineal.

7. RecursionError y los límites de la pila

Llegamos a una pregunta operativa importante: ¿qué pasa cuando una recursión

excede el límite de profundidad de Python? El intérprete lanza una excepción de la clase `RecursionError` —una subclase de `RuntimeError`— con el mensaje típico *maximum recursion depth exceeded*. Esto previene un *stack overflow* a nivel del sistema operativo, que tendría consecuencias mucho más graves: el proceso se aborta sin posibilidad de manejar la excepción, y los datos en RAM se pierden.

Veamos qué ocurre en la práctica:

```
import sys

def cuenta_descendente(n):
    if n == 0:
        return 0
    return cuenta_descendente(n - 1) + 1

print(sys.getrecursionlimit())    # 1000 (valor por defecto en CPyt

cuenta_descendente(997)           # funciona
cuenta_descendente(2000)          # RecursionError
# Traceback (most recent call last):
#   File "...", line 1, in <module>
#   File "...", line 4, in cuenta_descendente
#   ... [líneas repetidas]
# RecursionError: maximum recursion depth exceeded
```

El límite por defecto es 1000, pero la profundidad máxima alcanzable es ligeramente menor —hay frames de Python utilizados internamente por el intérprete. Por eso la prueba con 997 puede funcionar y con 998 puede fallar, dependiendo de la versión.

7.1. Modificar el límite (con cuidado)

```
import sys
```

```
sys.setrecursionlimit(10000)
cuenta_descendente(5000)           # ahora funciona

sys.setrecursionlimit(1000000)
cuenta_descendente(500000)
# Posible Segmentation fault: agotamos la pila REAL del SO,
# no la simulada de Python. El proceso se aborta sin traceback.
```

Por qué subir el límite es peligroso

El límite de recursión de Python es una *cota suave*: lo aplica el intérprete para abortar de manera ordenada, lanzando una excepción que el código puede capturar. La pila *real* del proceso, gestionada por el SO, es una *cota dura*: cuando se agota, el kernel envía una señal SIGSEGV (*segmentation fault*) que aborta el proceso sin remedio. El límite por defecto de 1000 está calibrado para que la cota suave se alcance mucho antes que la dura. Subirlo a 10 000 o 100 000 puede ser razonable; subirlo a un millón está pidiendo que el SO mate al proceso sin contemplaciones.

7.2. La conexión con la pila del SO

Recordemos lo que vimos en la Semana 7 sobre la pila del proceso:

- Cada proceso tiene una pila —un segmento de memoria virtual asignado por el kernel al crear el proceso.
- El tamaño por defecto en Linux es 8 MiB; puede consultarse con `ulimit -s` y modificarse con `ulimit -s <kib>`.
- Cada llamada de función consume un frame; el tamaño del frame depende de la cantidad de variables locales y de los argumentos.
- Cuando la pila intenta crecer más allá de su asignación, el kernel detecta el acceso a memoria no asignada y termina el proceso con SIGSEGV.

Un frame de Python es relativamente caro: aproximadamente 500-1000 bytes (entre el frame propio del intérprete y el frame correspondiente del runtime de C).

Esto explica por qué el límite por defecto es 1000: $1000 \text{ frames} \times 1 \text{ KiB} \approx 1 \text{ MiB}$, cómodamente dentro de los 8 MiB de la pila del SO. Subir el límite a 100 000 frames consumiría aproximadamente 100 MiB —diez veces más que el tamaño de la pila por defecto. De ahí el riesgo.

Mismo principio, dos niveles de protección

Es elegante observar que aquí actúan *dos sistemas de protección contra el desbordamiento*: el intérprete Python cuenta sus propios frames y aborta con una excepción manejable cuando los excede; el kernel del SO controla la pila del proceso y aborta con SIGSEGV cuando se intenta crecer más allá. Es la misma estructura que vimos en la Semana 8 con los archivos: el lenguaje impone validaciones a nivel de aplicación, y el SO impone validaciones a nivel del sistema. Las dos capas son complementarias: la suave es informativa, la dura es el límite físico.

8. El heap revisitado: memoria dinámica y sus peligros

Hasta aquí nos concentramos en el comportamiento de la *pila* durante la recursión. Pero los frames recursivos no son la única clase de memoria que un programa consume: cada vez que creamos una lista, una cadena, un objeto definido por el usuario, esa memoria no vive en la pila sino en el **heap**, esa región dinámica que se extiende hacia arriba mientras la pila crece hacia abajo.

Conexión con la Semana 7: el modelo de memoria de cuatro segmentos

En la Semana 7 estudiamos el modelo clásico de memoria de un proceso, dividido en cuatro segmentos: *texto* (código ejecutable), *datos* (variables globales y estáticas), *heap* (memoria dinámica que crece hacia arriba) y *pila* (frames de funciones que crece hacia abajo). Hasta esta semana habíamos focalizado en la pila como el escenario de la ejecución de funciones. Ahora le toca al heap: ese espacio donde viven los objetos cuyo tamaño o tiempo de vida no se conoce en tiempo de compilación.

8.1. Por qué necesitamos el heap

La pila tiene una característica restrictiva: el tamaño de cada frame debe conocerse en el momento de generar código. Si una función declara una variable local `int x`, el compilador sabe que ocupa 4 bytes y puede reservarlos en el frame. Pero si una función necesita una lista cuyo tamaño dependerá de un valor leído en tiempo de ejecución, ese tamaño no puede vivir en el frame: *el frame ya estará armado cuando ese valor se conozca*. La solución histórica es separar la memoria en dos regiones con disciplinas distintas:

- **Pila:** tamaños fijos conocidos al compilar; disciplina LIFO; asignación y liberación automáticas al entrar/salir de cada función; muy rápida (un decremento del puntero de pila).
- **Heap:** tamaños arbitrarios determinados en tiempo de ejecución; sin disciplina de orden de liberación; asignación y liberación explícitas o asistidas por el lenguaje; más costosa (búsqueda de un bloque libre del tamaño adecuado).

Conexión con la Semana 8: la lista de bloques libres

El heap se gestiona internamente con una *free list*: una lista enlazada de bloques disponibles, ordenados por dirección o por tamaño. Cuando se solicita memoria, el gestor recorre esa lista buscando un bloque suficiente; cuando se libera, el bloque se reinserta. Es exactamente la misma estructura que estudiamos en la Semana 8 para el manejo de bloques libres en el

sistema de archivos: el sistema operativo no inventa una solución nueva, reutiliza el mismo patrón algorítmico para problemas estructuralmente análogos —reservar y liberar regiones contiguas de un recurso compartido—.

8.2. Memoria dinámica en C: `malloc` y `free`

En C, la gestión del heap es *completamente manual*. El programador solicita memoria con `malloc` (memory allocate) y la devuelve con `free`. Conceptualmente:

```
// Reserva memoria para un arreglo de n enteros
int *arr = (int *) malloc(n * sizeof(int));

if (arr == NULL) {
    // El heap está lleno; malloc devolvió NULL
    fprintf(stderr, "Sin memoria\n");
    return 1;
}

// Usar arr[0], arr[1], ..., arr[n-1] ...

free(arr); // Devolver la memoria al heap
arr = NULL; // Higiene: invalidar el puntero
```

Esta libertad tiene tres errores clásicos —los *tres pecados capitales* de la memoria dinámica— cada uno con consecuencias potencialmente catastróficas:

8.3. Los tres pecados capitales de la memoria dinámica

Pecado 1: Memory leak (fuga de memoria)

Un *memory leak* ocurre cuando se asigna memoria con `malloc` pero nunca se libera con `free`. Si esto sucede dentro de un bucle o de una función llamada repetidamente, el heap del proceso crece indefinidamente. El síntoma típico es un programa cuyo consumo de memoria aumenta de forma monótona con el tiempo

de uso, hasta agotar la memoria física del sistema:

```
void procesar_archivo(const char *ruta) {
    char *buffer = (char *) malloc(1024);
    // ... usar buffer ...
    return; // ⚠ FUGA: nunca llamamos free(buffer)
}

// Si esta función se invoca un millón de veces,
// el proceso retiene 1 GiB de memoria innecesariamente.
```

Pecado 2: Dangling pointer (puntero colgante)

Un *dangling pointer* es un puntero que sigue apuntando a una dirección de memoria que ya fue liberada. Acceder a través de él es *comportamiento indefinido*: a veces la memoria aún contiene los datos antiguos y todo parece funcionar; otras veces ya fue reasignada a otro objeto y la lectura devuelve basura, o la escritura corrompe ese otro objeto. La impredecibilidad es lo más peligroso:

```
int *p = (int *) malloc(sizeof(int));
*p = 42;
free(p);
// p sigue conteniendo la dirección liberada (dangling)
printf("%d\n", *p); // ⚠ comportamiento indefinido
```

Pecado 3: Double free (doble liberación)

Llamar a `free` dos veces sobre el mismo puntero corrompe la estructura interna del gestor del heap (la free list). El programa puede no fallar inmediatamente, pero la próxima asignación probablemente devuelva memoria inválida o el siguiente `free` se cuelgue. Es uno de los errores más difíciles de diagnosticar porque la causa y el síntoma están separados en el tiempo:

```
int *p = (int *) malloc(sizeof(int));
free(p);
free(p); // ⚠ double free: corrompe el heap
```

⚠ Vulnerabilidades de seguridad

Los tres pecados capitales no son meras curiosidades académicas. Una porción significativa de las vulnerabilidades críticas de seguridad reportadas históricamente en software de sistema (sistemas operativos, navegadores, servidores) se origina en errores de manejo de memoria dinámica. *Use-after-free* y *heap buffer overflow* son nombres familiares para cualquier persona que siga reportes **CVE** (*Common Vulnerabilities and Exposures*: el catálogo público mantenido por la corporación MITRE en el que se registran y enumeran las vulnerabilidades de seguridad conocidas en software, asignándoles un identificador único de la forma CVE-AAAA-NNNN). Esta es una de las razones por las cuales lenguajes más modernos —Java, Python, Go, Rust— eligen distintas estrategias para automatizar o restringir la gestión del heap.

8.4. Diagrama de los tres pecados

El siguiente diagrama ilustra los tres errores clásicos en una vista comparada del heap:

Los tres pecados capitales del heap



Figura 8.1. Los tres errores clásicos de manejo manual de memoria dinámica en C/C++.

Estos tres errores tienen una característica común especialmente perniciosa: **el momento del error y el momento del síntoma están separados**. Un memory leak en una función se manifiesta horas después con un proceso lento o un kernel matando el proceso por consumo excesivo. Un dangling pointer puede leer datos que «parecen correctos» muchas veces antes de que la memoria sea reasignada y la lectura empieza a devolver basura. Un double free puede sobrevivir minutos hasta que el siguiente `malloc` elija el bloque corrupto. Esta separación causa-síntoma es lo que hace que estos errores sean famosos por la dificultad de su diagnóstico.

9. El recolector de basura de Python

Python toma una decisión radicalmente distinta a la de C: *el programador nunca llama a algo equivalente a `free`*. La memoria dinámica se libera automáticamente cuando el sistema determina que ya nadie la está usando. Este sistema automático se llama **recolector de basura** (garbage collector, GC) y combina dos mecanismos complementarios: *conteo de referencias* y *detección de ciclos*.

9.1. Conteo de referencias (reference counting)

Cada objeto en CPython lleva un contador interno: cuántas variables, atributos o elementos de colección están actualmente apuntando a él. Cuando una nueva referencia al objeto se crea, el contador sube; cuando una referencia se destruye, el contador baja. Cuando el contador llega a cero, el objeto es liberado *inmediatamente*:

```
import sys
```

```
x = [1, 2, 3]
```

```
# Hay una referencia al objeto lista: x
```

```
print(sys.getrefcount(x))    # 2 (la +1 es por el argumento de getr
```

```
y = x
```

```
# Ahora hay dos referencias: x e y
```

```
print(sys.getrefcount(x))    # 3

del x
# Una referencia menos; queda solo y
print(sys.getrefcount(y))    # 2

del y
# refcount llega a 0: la lista se libera automáticamente
```



Por qué `getrefcount(x)` devuelve uno más

El argumento de cualquier función —incluida `getrefcount`— cuenta como una referencia adicional mientras la función se ejecuta. Por eso, el primer ejemplo muestra 2 cuando «solo» `x` apunta al objeto: la 1 esperada más la del parámetro temporal de `getrefcount`.

El conteo de referencias tiene tres virtudes notables como mecanismo de gestión automática de memoria:

- **Determinismo:** el objeto se libera *en el momento exacto* en que su última referencia desaparece. No hay pausas impredecibles; los archivos se cierran cuando salen de scope, las conexiones a base de datos se devuelven al pool en el instante esperado.
- **Localidad temporal:** el objeto se libera mientras aún es probable que esté en caché, lo que ayuda al rendimiento del cache de CPU.
- **Simplicidad:** el algoritmo es trivial: incrementar/decrementar un entero y verificar si llegó a cero.

Pero tiene un agujero conceptual del que ni los autores originales pudieron escapar: *los ciclos de referencias*.

9.2. El problema de los ciclos

Consideremos dos objetos que se apuntan mutuamente:

```
a = []
b = []
a.append(b)    # a contiene a b
b.append(a)    # b contiene a a

# Estado de los contadores:
#   a -> refcount = 2 (apuntado por la variable a y por b[0])
#   b -> refcount = 2 (apuntado por la variable b y por a[0])

del a
del b
# Ahora:
#   ex-objeto-a -> refcount = 1 (aún apuntado por b[0])
#   ex-objeto-b -> refcount = 1 (aún apuntado por a[0])
# Ninguno llega a 0, pero NADIE puede acceder a ellos.
# Esto es una fuga de memoria con conteo de referencias puro.
```

El conteo de referencias por sí solo no detecta esta situación: ambos objetos tienen `refcount > 0`, pero son inalcanzables desde cualquier raíz del programa. Forman una *isla de basura*. Para resolver esto, CPython incorpora un segundo mecanismo: el *detector de ciclos generacional*.

9.3. Detección de ciclos: GC generacional

CPython mantiene una lista de objetos que *podrían* participar en ciclos — esencialmente, todos los objetos contenedores: listas, diccionarios, tuplas con contenedores, instancias de clases, etc.—. Periódicamente recorre esa lista en busca de grupos cuyas referencias internas se anulan unas a otras. La hipótesis del GC generacional es la **hipótesis generacional**:

💡 Hipótesis generacional

La gran mayoría de los objetos muere joven: variables temporales, resultados intermedios, datos de un bucle. Los pocos objetos que sobreviven a las primeras recolecciones tienden a sobrevivir mucho más tiempo. Por lo tanto, conviene revisar con mayor frecuencia los objetos jóvenes y con menor frecuencia los viejos.

CPython implementa esto con tres generaciones (numeradas 0, 1 y 2). Cada vez que se ejecuta una recolección en una generación, los objetos sobrevivientes son promovidos a la generación siguiente. La generación 0 se revisa con frecuencia; la 2, raramente:

```
import gc

# Umbrales por defecto: cada cuántas asignaciones se dispara
# una recolección en cada generación.
print(gc.get_threshold()) # (700, 10, 10)

# Forzar una recolección completa (las 3 generaciones)
basura_recolectada = gc.collect()
print(f"Objetos liberados: {basura_recolectada}")

# Estadísticas por generación
print(gc.get_stats())
```

Los umbrales por defecto significan: tras 700 asignaciones netas (asignaciones menos liberaciones), se dispara una recolección de la generación 0. Cada 10 recolecciones de la generación 0, se dispara una de la 1. Cada 10 de la 1, una de la 2. Esta jerarquía hace que el costo amortizado del GC sea bajo.

9.4. El algoritmo de detección: *mark and sweep* sobre los refcounts

El algoritmo concreto que usa CPython para detectar ciclos es ingenioso. Para cada generación revisada:

1. **Copia de los refcounts.** Para cada objeto candidato, se hace una copia de su `refcount` en un campo auxiliar (lo llamamos `gc_refs`).
2. **Resta de referencias internas.** Se recorre el grafo de objetos y, por cada referencia entre dos objetos del conjunto, se decrementa el `gc_refs` del referido. El resultado: cada objeto queda con un `gc_refs` que cuenta solo las referencias *externas* al conjunto candidato.
3. **Marcado de alcanzables.** Cualquier objeto con `gc_refs` > 0 tiene al menos una referencia externa, por lo tanto es alcanzable. Se marca como vivo, y se propaga la marca a todos los objetos accesibles desde él.
4. **Barrido.** Los objetos que quedaron sin marcar son inalcanzables: forman ciclos que ya no participan del programa. Se liberan.

Detección de ciclos: lo alcanzable y lo isla

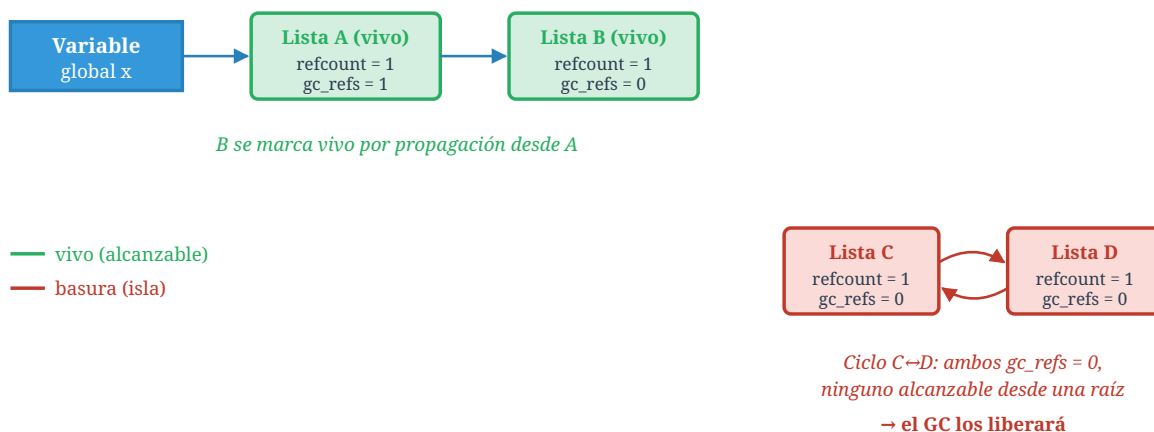


Figura 9.1. La detección de ciclos identifica grupos de objetos cuyas referencias se cierran sobre sí mismas y son inalcanzables desde cualquier raíz del programa. Aunque sus `refcounts` no son cero, el GC los libera.

9.5. Comparación de estrategias de gestión del heap

Aspecto	C/C++ manual	Python (refcount + GC)	Java/Go (GC tracing puro)
Liberación	Explícita: <code>free</code>	Automática: refcount o ciclo	Automática: ciclos periódicos

Aspecto	C/C++ manual	Python (refcount + GC)	Java/Go (GC tracing puro)
Determinismo del momento	Total (lo decide el programador)	Alto (refcount es inmediato)	Bajo (GC corre cuando le conviene)
Velocidad	Máxima (sin overhead)	Buena (overhead por incremento/decremento)	Muy buena en régimen, con pausas
Memory leaks	Posibles (olvido de free)	Posibles solo en ciclos no detectados	Casi imposibles
Dangling pointers	Posibles	Imposibles por diseño	Imposibles por diseño
Pausas (stop-the-world)	Ninguna	Cortas (en detección de ciclos)	Variables, optimizable

💡 Implicaciones para el diseño de programas Python

Aunque Python libera memoria automáticamente, hay tres situaciones donde el programador debe ser consciente del modelo: (a) los archivos y conexiones de red requieren cierre explícito vía `with` —que estudiamos en la Semana 6—, porque el cierre se considera un efecto observable y no debe depender del momento incierto en que el GC libere el objeto; (b) las estructuras de datos con referencias mutuas (árboles bidireccionales, grafos) generan ciclos que el GC eventualmente resolverá, pero conviene romperlos manualmente cuando se sabe que el subgrafo ya no se usa; (c) en aplicaciones con restricciones estrictas de latencia, las pausas del detector de ciclos —aunque sean breves— pueden importar, y existe la API `gc.disable()` / `gc.collect()` para controlarlas.

10. Mirada holística: gramáticas formales y árboles de derivación

La recursión no es solo una técnica de programación: es la herramienta conceptual fundamental para describir *lenguajes*. Cada vez que escribimos código Python, hay un parser detrás leyendo nuestros caracteres, agrupándolos en

tokens, y reconociendo recursivamente la estructura de cada expresión. En esta sección exploramos cómo los conceptos de la teoría de lenguajes formales conectan con todo lo visto en la asignatura.

Conexión con la Semana 1 y la Semana 6

En la *Semana 1* distinguimos lenguajes de máquina, ensambladores, lenguajes de alto nivel y lenguajes naturales. En la *Semana 6* diferenciamos compiladores e intérpretes: dos arquitecturas distintas para traducir o ejecutar código fuente. En ambos casos, quedó pendiente una pregunta fundamental: *¿cómo se especifica formalmente la sintaxis de un lenguaje, de modo que un programa pueda decidir si una secuencia de caracteres es un programa válido?* La respuesta son las gramáticas formales, y su uso práctico está profundamente entrelazado con la recursión.

10.1. Qué es una gramática formal

Una gramática formal es un conjunto finito de reglas que especifican qué cadenas pertenecen a un lenguaje. Formalmente, una gramática es una tupla $G = (V, \Sigma, P, S)$, donde:

- V es el conjunto de **símbolos no terminales** (categorías sintácticas, como «expresión» o «sentencia»).
- Σ es el conjunto de **símbolos terminales** (los caracteres o tokens del lenguaje, como `+`, `123`, `if`).
- P es el conjunto de **producciones**: reglas de la forma $A \rightarrow \alpha$ que reemplazan un no-terminal por una secuencia de terminales y no-terminales.
- S es el **símbolo inicial**, el no-terminal por el cual comienza toda derivación.

Las *gramáticas libres de contexto* (CFG, context-free grammars) son las que más nos interesan: en ellas, el lado izquierdo de cada producción es un único no-terminal. Casi todos los lenguajes de programación se especifican mediante CFGs (con pequeñas extensiones). Su importancia es que existen algoritmos eficientes para decidir, dada una cadena, si pertenece al lenguaje generado por la gramática —y los algoritmos más naturales para hacerlo son *recursivos*.

10.2. Notación BNF y EBNF

Para escribir gramáticas se utiliza desde la década de 1960 la notación **BNF** (Backus-Naur Form), introducida originalmente para especificar Algol 60. La notación usa <nombre> para no-terminales y ::= para indicar «se define como». Veamos una gramática BNF para expresiones aritméticas simples:

```
<expresion> ::= <termino>
              | <expresion> + <termino>
              | <expresion> - <termino>

<termino>    ::= <factor>
              | <termino> * <factor>
              | <termino> / <factor>

<factor>     ::= <numero>
              | ( <expresion> )

<numero>     ::= <digito>
              | <numero> <digito>

<digito>     ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Observemos lo central: *la definición de <expresion> se refiere a sí misma*. Y la de <termino> también. Y <factor> contiene <expresion> entre paréntesis. La gramática es *recursiva*: define expresiones complejas en términos de expresiones más simples del mismo tipo. Es exactamente la misma estructura que estudiamos en las secciones anteriores: caso base (un número) más caso recursivo (una expresión más un término).

La notación **EBNF** (Extended BNF) agrega tres operadores que abrevian patrones comunes:

- { X } — cero o más repeticiones de X (análogo a un bucle `while`).
- [X] — cero o una aparición de X (opcional).
- X | Y — alternativa.

Con EBNF, la gramática anterior se vuelve más legible:

```
expresion ::= termino { ( + | - ) termino }
```

```
termino    ::= factor { ( * | / ) factor }

factor     ::= numero
            | ( expresion )

numero     ::= digito { digito }

digito     ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

EBNF es la notación que usan hoy la mayoría de las especificaciones de lenguajes (incluida la *Python Language Reference*). Cuando se documenta un lenguaje, lo que se entrega son sus reglas EBNF como *especificación formal*: esto es, el contrato que cualquier intérprete o compilador debe respetar.

10.3. Árbol de derivación

Cuando una gramática reconoce una cadena, hay un objeto natural asociado: el **árbol de derivación** (parse tree, árbol sintáctico). Cada nodo interno es un no-terminal; las hojas son terminales; los hijos de un nodo se determinan aplicando una producción. El árbol explica *cómo* se obtiene la cadena a partir del símbolo inicial.

Tomemos la expresión $3 + 4 * 5$. Su árbol de derivación según la gramática anterior:

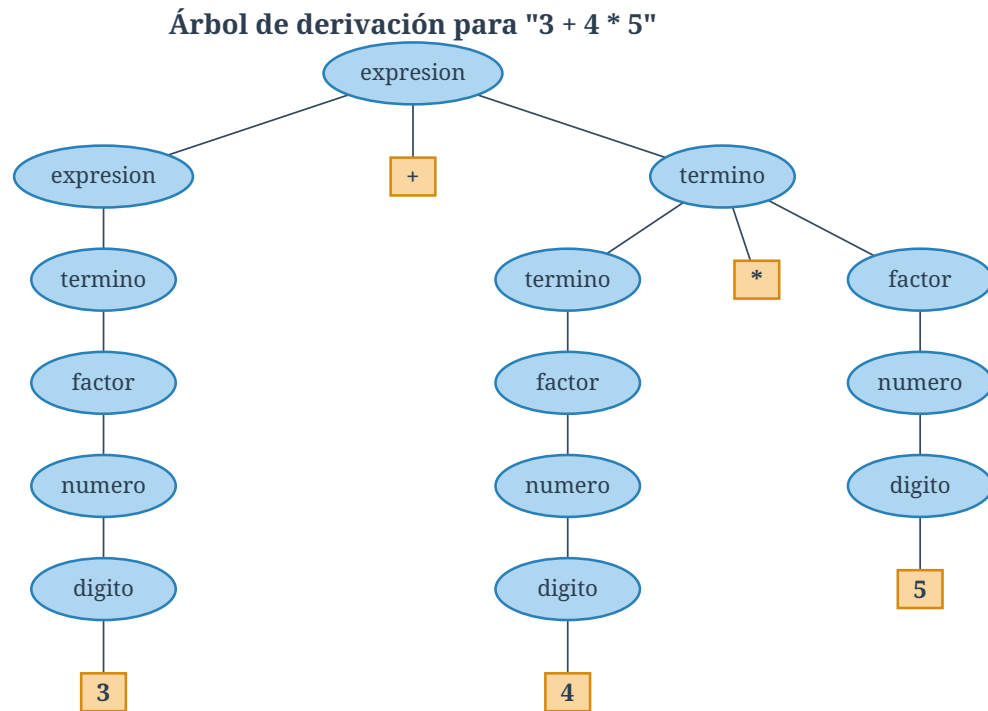


Figura 10.1. Árbol de derivación para la expresión $3 + 4 * 5$. La estructura del árbol revela que la multiplicación se evalúa primero (es un subárbol del operando derecho de la suma), porque la gramática define *termino* dentro de *expresion*: es la jerarquía gramatical la que determina la precedencia de operadores.

Observación importante: la *precedencia de operadores* no es una decisión separada que el parser tome después; *es la estructura misma de la gramática*. Como *termino* está "más adentro" que *expresion* en la jerarquía, el operador $*$ queda más próximo a las hojas y se evalúa antes que $+$. Cambiar la precedencia sería cambiar la gramática.

10.4. Parsers de descenso recursivo

La técnica más natural para implementar un parser sobre una CFG se llama **descenso recursivo** (recursive descent): se escribe una función por cada no-terminal, y cada función se llama a las demás exactamente según las producciones de la gramática. Es un mapeo uno-a-uno entre la especificación EBNF y el código:

```
# Parser de descenso recursivo para expresiones aritméticas.  
# Sigue exactamente la gramática EBNF de 10.2.
```

```
def parse_expresion(tokens, i):
```

```
"""
```

Reconoce una expresión a partir de la posición *i*.

Precondición: *tokens* es una lista no vacía; $0 \leq i < \text{len}(\text{tokens})$

Postcondición: retorna (*valor*, *j*) donde *valor* es el resultado y *j* es la posición del primer token no consumido.

```
"""
```

```
valor, i = parse_termino(tokens, i)
```

```
while i < len(tokens) and tokens[i] in ('+', '-');
```

```
    op = tokens[i]
```

```
    derecha, i = parse_termino(tokens, i + 1)
```

```
    valor = valor + derecha if op == '+' else valor - derecha
```

```
return valor, i
```

```
def parse_termino(tokens, i):
```

```
    valor, i = parse_factor(tokens, i)
```

```
    while i < len(tokens) and tokens[i] in ('*', '/');
```

```
        op = tokens[i]
```

```
        derecha, i = parse_factor(tokens, i + 1)
```

```
        valor = valor * derecha if op == '*' else valor / derecha
```

```
    return valor, i
```

```
def parse_factor(tokens, i):
```

```
    if tokens[i] == '(':
```

```
        valor, i = parse_expresion(tokens, i + 1)
```

```
        # Esperamos paréntesis de cierre
```

```
        return valor, i + 1
```

```
    return int(tokens[i]), i + 1
```

```
# Uso:
```

```
tokens = ['3', '+', '4', '*', '5']
```

```
resultado, _ = parse_expresion(tokens, 0)
```

```
print(resultado)    # 23 (la precedencia es respetada por la gramática)
```

Las llamadas mutuamente recursivas entre `parse_expresion`, `parse_termino` y `parse_factor` son el reflejo exacto de las producciones de la gramática. La pila de

ejecución del parser, durante el reconocimiento de una expresión compleja, es *isomorfa al árbol de derivación*: los frames apilados corresponden a los nodos del árbol que se están explorando.

CPython usa parser de descenso recursivo

Históricamente, CPython usó un parser LL(1) generado automáticamente. Desde la versión 3.9 utiliza un parser PEG (Parsing Expression Grammar) que también es esencialmente un descenso recursivo, con backtracking implícito. La gramática de Python está especificada formalmente en el archivo `Grammar/python.gram` del repositorio fuente, en notación PEG (una variante de EBNF). Cada vez que ejecutamos un programa Python, la primera tarea del intérprete es invocar este parser recursivo sobre el código fuente. Es decir: la recursión es lo primero que sucede al correr nuestro código, antes incluso de que ninguna línea se ejecute.

10.5. La jerarquía de Chomsky en perspectiva

Las gramáticas libres de contexto son uno de cuatro niveles en una jerarquía propuesta por Noam Chomsky en 1956. De más expresiva a menos expresiva:

Tipo	Nombre	Reconocida por	Ejemplos
Tipo 0	Irrestringidas	Máquinas de Turing	Lenguajes computables
Tipo 1	Sensibles al contexto	Autómatas linealmente acotados	Algunas verificaciones de tipos
Tipo 2	Libres de contexto (CFG)	Autómatas de pila (PDA)	Sintaxis de la mayoría de lenguajes
Tipo 3	Regulares	Autómatas finitos (DFA)	Expresiones regulares, tokens

El detalle conceptualmente más significativo: las CFG son reconocidas por *autómatas de pila*. La pila no es un detalle de implementación sino una necesidad teórica para reconocer estructuras anidadas como paréntesis balanceados. *La pila es el correlato matemático de la recursividad*: un parser recursivo y un autómata de pila son la misma cosa expresadas con vocabularios distintos.

La pila vuelve a aparecer

En la Semana 4 introdujimos la pila de ejecución como mecanismo del SO para gestionar llamadas a funciones. En la Semana 7 la analizamos en detalle como uno de los segmentos del modelo de memoria. En esta semana la usamos para entender la recursión como recursión efectiva sobre la pila. Y aquí, en una vuelta de tuerca teórica, descubrimos que la pila no es solo el mecanismo concreto: es el objeto matemático cuya capacidad expresiva define exactamente qué lenguajes son los CFG. *La pila no es solo cómo se implementa la recursividad: es lo que la recursividad es.*

11. Síntesis y mirada hacia adelante

Repasemos lo recorrido en la semana, conectando explícitamente los conceptos:

- **Recursividad como técnica algorítmica** (Secciones 1–5): el patrón caso base + caso recursivo permite resolver problemas auto-similares. La pila de ejecución —estudiada desde la Semana 4— es lo que permite que las llamadas anidadas mantengan estados independientes.
- **Análisis comparativo iteración vs. recursión** (Secciones 4 y 6): la elección no es ideológica sino contextual. La recursividad de cola es un ideal teórico que Python deliberadamente no optimiza, por razones de transparencia para el debugger.
- **Los límites de la pila** (Sección 7): `RecursionError` protege al intérprete de un stack overflow del SO. Dos capas de protección actuando en concierto, análogas a la separación entre validaciones del lenguaje y validaciones del SO que vimos en la Semana 8.
- **El heap y sus pecados** (Sección 8): la memoria dinámica del proceso, gestionada con la misma estructura de free list que el sistema de archivos de la Semana 8. Tres errores clásicos —leak, dangling, double free— son el origen histórico de muchas vulnerabilidades de seguridad.
- **El recolector de basura** (Sección 9): la respuesta de Python a esos errores. El conteo de referencias resuelve el caso común; la detección generacional de ciclos resuelve el caso patológico. La hipótesis generacional permite amortizar el costo.

- **Gramáticas formales** (Sección 10): el mismo patrón recursivo que usábamos para algoritmos describe la sintaxis de los lenguajes. La pila aparece en el corazón teórico de la jerarquía de Chomsky: autómatas de pila reconocen las CFG. Cada vez que ejecutamos código Python, la primera operación del intérprete es un parser recursivo de descenso.

Hay una idea unificadora que atraviesa todo lo visto: *los problemas auto-similares se expresan, almacenan y procesan recursivamente*. La recursión no es una técnica entre otras; es la respuesta natural a una clase amplia de fenómenos — estructuras de datos jerárquicas, algoritmos divide-y-conquista, sintaxis de lenguajes, sistemas de archivos— que comparten una propiedad geométrica: cada parte tiene la misma forma que el todo.



El camino recorrido y lo que viene

Los conceptos de esta semana se apoyan sobre toda la arquitectura conceptual que venimos construyendo, y a su vez son la base para los próximos temas:

Lo que conecta hacia atrás:

- **Semana 1 (lenguajes y arquitectura Von Neumann):** los niveles de lenguaje y las gramáticas que los formalizan; el operador $n \gg 1$ en potencia rápida como conexión con la ALU.
- **Semana 4 (funciones y pila):** la pila de ejecución que ahora analizamos en detalle como sustento de la recursividad.
- **Semana 5 (búsqueda binaria):** reformulada esta semana como recursión de cola, prototipo de divide y vencerás.
- **Semana 6 (compiladores e intérpretes, merge sort):** el merge sort como ejemplo canónico de divide y vencerás; la sintaxis del lenguaje que ahora sabemos describir formalmente.
- **Semana 7 (modelo de memoria):** los cuatro segmentos —pila y heap entre ellos— revisitados desde una perspectiva más rica.
- **Semana 8 (archivos binarios y free list):** el patrón de gestión de bloques libres reaparece en el heap; las dos capas de protección se mantienen.

Lo que se prepara hacia adelante:

- **Semana 10 (diccionarios y conjuntos):** las primeras estructuras de datos no lineales del curso. Su implementación interna —tablas hash— usará el heap intensivamente y planteará preguntas finas sobre el costo amortizado de las operaciones.
- **Semana 11 (estructuras lineales: pilas y colas):** implementaremos explícitamente la estructura abstracta de pila que aquí usamos como caja negra. Veremos que hay múltiples implementaciones (lista dinámica, lista enlazada) con propiedades distintas.
- **Semana 12 (estructuras enlazadas, listas y árboles):** los árboles serán el escenario natural de la recursividad. Recorridos in-orden, pre-orden y post-orden son recursivos por excelencia, e iluminarán retrospectivamente lo que hicimos esta semana.
- **Semanas 13–16 (algoritmos avanzados y proyecto integrador):** programación dinámica, backtracking, grafos —todos basados en los conceptos de recursividad establecidos esta semana—.

La recursividad es, posiblemente, el primer concepto del curso que requiere un cambio cualitativo en la forma de pensar sobre los programas. Si hasta aquí pensábamos en términos de pasos secuenciales, ahora pensamos en términos de auto-similaridad. Cuando se domina, se descubre que muchos problemas que parecían arduos en versión iterativa son casi triviales en versión recursiva. La inversa también es cierta. Saber elegir es la marca del programador maduro.

Recursividad y manejo de memoria dinámica

Semana 9 — Desarrollo de Contenidos — Algoritmia y Programación — Carrera de Informática — FIUBA